2026-10-21 MySQL表的约束：下

- 2. MySQL 表的约束（下）
  - 2.1 自增长：AUTO_INCREMENT
    - 2.1.1 AUTO_INCREMENT 的作用
    - 2.1.2 自增长必须满足的条件
    - 2.1.3 显式写入较大编号
    - 2.1.4 手动设置初始值
    - 2.1.5 获取刚刚生成的自增值
  - 2.2 唯一键：UNIQUE KEY
    - 2.2.1 为什么还需要唯一键
    - 2.2.2 唯一键为什么允许 NULL
    - 2.2.3 主键与唯一键的区别
    - 2.2.4 复合唯一键
    - 2.2.5 唯一键设计建议
  - 2.3 外键：FOREIGN KEY
    - 2.3.1 外键的两层含义
    - 2.3.2 主表与从表
    - 2.3.3 创建班级表和学生表
    - 2.3.4 合法插入与非法插入
    - 2.3.5 删除主表记录时会发生什么
    - 2.3.6 外键约束的价值
  - 2.4 综合案例：商城表结构
    - 2.4.1 业务需求
    - 2.4.2 需求到约束的映射
    - 2.4.3 创建数据库和三张表
    - 2.4.4 表间关系
    - 2.4.5 综合案例的工程改进
  - 2.5 第二部分小结
    - 2.5.1 三类约束对照
    - 2.5.2 表约束的设计顺序
    - 2.5.3 核心结论

2. MySQL 表的约束（下）

2.1 自增长：AUTO_INCREMENT

2.1.1 AUTO_INCREMENT 的作用

`AUTO_INCREMENT` 用于自动生成整数编号。插入数据时，如果该字段没有显式赋值，MySQL 会从当前自增计数器中取得下一个值；新增值通常比表中已有的最大编号更大。

自增长最适合与主键搭配，形成与具体业务无关的逻辑主键。例如学生、商品、订单都可以拥有一个自动生成的编号，而不用每次由程序手工寻找下一个可用值。

测试表：

```
CREATE TABLE IF NOT EXISTS t16 (
    id INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(20) NOT NULL
);
```

```
mysql> desc t16;
+-------+------------------+------+-----+---------+----------------+
| Field | Type             | Null | Key | Default | Extra          |
+-------+------------------+------+-----+---------+----------------+
| id    | int(10) unsigned | NO   | PRI | NULL    | auto_increment |
| name  | varchar(20)      | NO   |     | NULL    |                |
+-------+------------------+------+-----+---------+----------------+
2 rows in set (0.00 sec)
```

连续插入时，只提供 `name`，`id` 会自动生成：

```
INSERT INTO t16(name) VALUES('a');
INSERT INTO t16(name) VALUES('b');
SELECT * FROM t16;
```

```
+----+------+
| id | name |
+----+------+
|  1 | a    |
|  2 | b    |
+----+------+
```

```
mysql> insert into t16 (name) values ('a');
Query OK, 1 row affected (0.01 sec)

mysql> insert into t16 (name) values ('b');
Query OK, 1 row affected (0.00 sec)

mysql> insert into t16 (name) values ('c');
Query OK, 1 row affected (0.00 sec)

mysql> select * from t16;
+----+------+
| id | name |
+----+------+
|  1 | a    |
|  2 | b    |
|  3 | c    |
+----+------+
3 rows in set (0.00 sec)
```

`AUTO_INCREMENT` 的价值不只是少写一个编号，更重要的是让每条新记录自动获得一个不重复、非空的标识，调用方不必手工维护计数器。

2.1.2 自增长必须满足的条件

一个字段要声明为自增长，需要满足以下条件：

- 字段必须是整数类型。
- 字段必须参与索引，`Key` 栏不能为空。
- 一张表最多只能有一个 `AUTO_INCREMENT` 字段。
- 实际设计中最常见的形式是“主键加自增长”，但索引条件并不等于必须显式声明 `PRIMARY KEY`。
- 自增长字段通常还应与 `NOT NULL` 一起使用，保证生成或写入的编号完整。

这里的“必须有索引”是数据库执行自增取号的条件。索引本质上是一种独立、有序的辅助数据结构，保存着数据值及其对应行的定位信息。可以把它类比成图书目录：数据库先查找目录，再快速定位到真正需要访问的数据行。

自增长与索引搭配后，数据库能够高效地找到当前计数状态，并把新值写入对应位置。后续学习索引时，还会进一步讨论聚簇索引、普通索引和唯一索引的底层结构。

2.1.3 显式写入较大编号

如果插入时显式给了编号，MySQL 会采用这个值；当它大于当前计数时，后续自动编号通常从这个较大值继续向后增长。

例如：

```
INSERT INTO t16(id, name) VALUES(100, 'manual');
INSERT INTO t16(name) VALUES('after-manual');
SELECT * FROM t16;
```

后一条未指定编号的记录通常会得到 `101`：

```
+-----+--------------+
| id  | name         |
+-----+--------------+
|   1 | a            |
|   2 | b            |
| 100 | manual       |
| 101 | after-manual |
+-----+--------------+
```

```
mysql> insert into t16 (id,name) values (1000,'d');
Query OK, 1 row affected (0.00 sec)

mysql> insert into t16 (name) values ('e');
Query OK, 1 row affected (0.00 sec)

mysql> select * from t16;
+------+------+
| id   | name |
+------+------+
|    1 | a    |
|    2 | b    |
|    3 | c    |
| 1000 | d    |
| 1001 | e    |
+------+------+
5 rows in set (0.00 sec)
```

因此，显式插入一个较大值会抬高后续自动编号的起点。但自增长并不保证编号连续：

- 插入失败或事务回滚可能消耗计数。
- 删除旧记录后，编号通常不会立刻回收再用。
- 并发插入和显式编号可能使编号之间出现空隙。
- 服务重启后的具体计数器状态还与存储引擎和 MySQL 版本有关。

所以自增长只应被视为“自动生成标识”的机制，不应被当成“连续、无缺口编号”的业务保证。

2.1.4 手动设置初始值

建表时可以通过表选项指定初始值：

```sql
CREATE TABLE t17 (
    id INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(20) NOT NULL
) AUTO_INCREMENT = 500;
```

向空表插入第一条未指定编号的记录时，通常会从 `500` 开始：

```
INSERT INTO t17(name) VALUES('first');
SELECT * FROM t17;
```

```
+-----+-------+
| id  | name  |
+-----+-------+
| 500 | first |
+-----+-------+
```

```
mysql> create table t17(
    -> id int unsigned primary key auto_increment,
    -> name varchar(20) not null
    -> )auto_increment=500;
Query OK, 0 rows affected (0.01 sec)

mysql> show create table t17 \G;
*************************** 1. row ***************************
       Table: t17
Create Table: CREATE TABLE `t17` (
  `id` int(10) unsigned NOT NULL AUTO_INCREMENT,
  `name` varchar(20) NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=500 DEFAULT CHARSET=utf8
1 row in set (0.01 sec)
```

设置初始值适合商品编号、订单批次等有预留区间的场景，但不应依赖编号区间来表达业务分类。业务分类应使用独立字段描述，避免编号规则变化时破坏历史语义。

2.1.5 获取刚刚生成的自增值

插入完成后，可以使用 `LAST_INSERT_ID()` 获取当前连接中最后一次由 `AUTO_INCREMENT` 生成的编号：

```
SELECT LAST_INSERT_ID();
```

如果一次插入多行，该函数返回本次批量插入中第一个自动生成的编号。例如批量插入得到 `3、4、5`，函数返回 `3`，后续编号可以据此推算。

`LAST_INSERT_ID()` 反映的是当前数据库连接最近一次生成的自增值。多连接并发写入时，不同连接各自维护自己的结果，不能把它理解为整张表当前最大的 `id`。

2.2 唯一键：UNIQUE KEY

2.2.1 为什么还需要唯一键

一张表最多只能有一个主键，但实际业务中经常有多个字段需要保证不重复。例如：

- 学生编号是主键。
- 学生手机号也不能重复。
- 学生邮箱同样不能重复。

`UNIQUE KEY` 就是用来补充这类业务唯一性要求的约束。它既能约束单列，也能约束多个字段的组合。

示例：

```
CREATE TABLE stu (
    id CHAR(20) UNIQUE COMMENT '这是一个学生的唯一键',
    name VARCHAR(32) NOT NULL
);
```

```
mysql> desc stu;
+-------+-------------+------+-----+---------+-------+
| Field | Type        | Null | Key | Default | Extra |
+-------+-------------+------+-----+---------+-------+
| id    | char(20)    | YES  | UNI | NULL    |       |
| name  | varchar(32) | NO   |     | NULL    |       |
+-------+-------------+------+-----+---------+-------+
2 rows in set (0.01 sec)
```

查看表结构时，`Key` 栏会出现 `UNI`，表示该列上建立了唯一约束：

```
DESC stu;
```

```
+-------+-------------+------+-----+---------+-------+
| Field | Type        | Null | Key | Default | Extra |
+-------+-------------+------+-----+---------+-------+
| id    | char(20)    | YES  | UNI | NULL    |       |
| name  | varchar(32) | NO   |     | NULL    |       |
+-------+-------------+------+-----+---------+-------+
```

插入不同的 `id` 可以成功：

```
INSERT INTO stu(id, name) VALUES('01', 'aaa');
```

再次插入相同的 `id` 会被拒绝：

```
INSERT INTO stu(id, name) VALUES('01', 'bbb');
```

```
ERROR 1062 (23000): Duplicate entry '01' for key 'id'
```

2.2.2 唯一键为什么允许 NULL

唯一键允许字段为空，而且多个 `NULL` 通常不会彼此冲突。原因是 `NULL` 表示“值未知”或“没有值”，MySQL 不会把两个未知值判定为业务上的重复项。

```
INSERT INTO stu(id, name) VALUES(NULL, 'bbb');
INSERT INTO stu(id, name) VALUES(NULL, 'ccc');
```

两条记录都可以存在：

```
+------+------+
| id   | name |
+------+------+
| 01   | aaa  |
| NULL | bbb  |
| NULL | ccc  |
+------+------+
```

这不表示唯一键没有价值，而是说明它只约束“确实存在且可比较的值”。如果业务要求该字段必须填写，应同时声明 `UNIQUE NOT NULL`：唯一性负责去重，非空性负责禁止缺失。

还需要区分空字符串与 `NULL`：

- `''` 是一个长度为 0 的真实值。
- 多个 `''` 在单列唯一键中会被视为重复。
- `NULL` 才是未赋值或未知，通常可重复出现。

2.2.3 主键与唯一键的区别

对比项	`PRIMARY KEY`	`UNIQUE KEY`
是否允许 `NULL`	不允许	允许，除非额外声明 `NOT NULL`
数量	每张表最多一个主键	可以有多个唯一键
主要目标	唯一、非空地标识一行	保证具体业务字段或组合不重复
能否由多列组成	可以，形成复合主键	可以，形成复合唯一键
`DESC` 中的 `Key`	`PRI`	`UNI`
常见例子	学生内部编号、订单编号	手机号、邮箱、员工工号

主键和唯一键不是互相替代的关系。学生可以用内部 `id` 作为主键，同时把手机号和邮箱分别设为唯一键。这样既保留了稳定、简洁的行标识，也表达了业务字段不能重复的规则。

课堂体会可以概括为：

- 主键更偏向“标识唯一性”，用于确定记录。
- 唯一键更偏向“业务唯一性”，用于防止重复信息。
- 两者共同提高约束力，防止程序员误操作。
- 唯一键也可以设置为 `NOT NULL`，但它仍然不是主键。

2.2.4 复合唯一键

当单个字段可以重复、但某组字段不能重复时，可以使用复合唯一键：

```
CREATE TABLE course_selection (
    student_id INT UNSIGNED,
    course_id INT UNSIGNED,
    score TINYINT UNSIGNED,
    UNIQUE KEY uk_student_course (student_id, course_id)
);
```

这里允许一个学生选择多门课，也允许一门课被多个学生选择，但不允许同一个学生重复选择同一门课。唯一性作用于 `(student_id, course_id)` 的整组值。

2.2.5 唯一键设计建议

- 优先使用与业务无关、稳定且非空的字段作为主键。
- 对于手机号、邮箱、员工工号等业务编号，可以建立唯一键。
- 业务要求字段必须存在时，使用 `UNIQUE NOT NULL`，不要只依赖唯一键。
- 唯一性约束会建立对应索引，既要考虑数据正确性，也要考虑写入和维护成本。
- 不要把“唯一”与“主键”混为一谈；选择哪一个取决于这个字段承担的是行标识还是业务去重职责。

2.3 外键：FOREIGN KEY

2.3.1 外键的两层含义

外键同时表达两件事：

1. 表与表之间存在关联关系。
2. 这种关联关系具有数据库层面的约束。

只有 `class_id` 字段而没有外键时，两张表只是在命名上看起来相关。数据库并不知道学生表中的 `class_id` 必须指向班级表中的真实班级，这种写法可以称为“有外键之名，没有外键之实”。

例如，班级表中只有 `10、20` 两个班，学生表中仍可能插入 `class_id = 30`，形成一个不存在班级的学生。数据表面完整，业务关系却已经断裂。

外键的本质是把关联检查交给 MySQL：应用插入、更新或删除相关数据时，数据库主动检查引用关系，不符合规则就拒绝操作。

2.3.2 主表与从表

外键场景中通常有两个角色：

- 主表：被引用的表，提供主键或唯一键。
- 从表：持有外键的表，通过外键引用主表的某个主键或唯一键。

外键约束定义在从表上，基本语法是：

```
FOREIGN KEY (从表字段) REFERENCES 主表(被引用字段)
```

使用外键时需要注意：

- 必须先创建主表，再创建从表。
- 被引用字段必须是主键或唯一键。
- 外键值必须在主表中存在，或者为 `NULL`。
- 只有允许 `NULL` 的外键列才能保存未分配关系。
- 两个字段的数据类型和字符集通常应兼容，避免比较或引用失败。

2.3.3 创建班级表和学生表

先创建班级表：

```
CREATE TABLE myclass (
    id INT PRIMARY KEY,
    name VARCHAR(30) NOT NULL COMMENT '班级名'
);
```

再创建学生表，并把 `class_id` 设为外键：

```
CREATE TABLE IF NOT EXISTS student (
    id INT UNSIGNED PRIMARY KEY,
    name VARCHAR(20) NOT NULL,
    telphone VARCHAR(32) UNIQUE KEY,
    class_id INT,
    FOREIGN KEY (class_id) REFERENCES myclass(id)
);
```

```
mysql> desc student;
+----------+------------------+------+-----+---------+-------+
| Field    | Type             | Null | Key | Default | Extra |
+----------+------------------+------+-----+---------+-------+
| id       | int(10) unsigned | NO   | PRI | NULL    |       |
| name     | varchar(20)      | NO   |     | NULL    |       |
| telphone | varchar(32)      | YES  | UNI | NULL    |       |
| class_id | int(11)          | YES  | MUL | NULL    |       |
+----------+------------------+------+-----+---------+-------+
4 rows in set (0.01 sec)
```

```
mysql> show create table student\G;
*************************** 1. row ***************************
       Table: student
Create Table: CREATE TABLE `student` (
  `id` int(10) unsigned NOT NULL,
  `name` varchar(20) NOT NULL,
  `telphone` varchar(32) DEFAULT NULL,
  `class_id` int(11) DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `telphone` (`telphone`),
  KEY `class_id` (`class_id`),
  CONSTRAINT `student_ibfk_1` FOREIGN KEY (`class_id`) REFERENCES `class` (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8
1 row in set (0.00 sec)

ERROR:
No query specified
```

学生表同时使用了多种约束：

- `id` 是非空且唯一的主键。
- `name` 不能为空。
- `telphone` 是业务唯一键。
- `class_id` 是外键，负责引用真实班级。

2.3.4 合法插入与非法插入

先向班级表写入两个真实班级：

```sql
INSERT INTO myclass VALUES
(10, 'C++大牛班'),
(20, 'Java大神班');
```

再写入引用这两个班级的学生：

```sql
INSERT INTO student VALUES
(100, '张三', '13800000001', 10),
(101, '李四', '13800000002', 20);
```

这两条数据都能通过外键检查。

如果尝试插入一个不存在的班级编号：

```
INSERT INTO student VALUES
(102, '王五', '13800000003', 30);
```

MySQL 会拒绝：

```
ERROR 1452 (23000): Cannot add or update a child row:
a foreign key constraint fails
```

如果学生暂时还没有分班，可以插入 `NULL`：

```
INSERT INTO student VALUES
(102, '王五', '13800000003', NULL);
```

这条数据可以成功，因为“尚未分配班级”与“引用一个不存在的班级”是两种不同状态。

2.3.5 删除主表记录时会发生什么

外键还会限制主表记录的删除。默认情况下，如果某个班级仍被学生引用，直接删除这个班级会被阻止：

```
DELETE FROM myclass WHERE id = 10;
```

```
ERROR 1451 (23000): Cannot delete or update a parent row:
a foreign key constraint fails
```

要合法删除班级，通常需要先处理引用它的学生数据：

- 删除对应学生记录。
- 修改这些学生的 `class_id`，改为另一个真实班级。
- 把学生的 `class_id` 改为 `NULL`，前提是该列允许空。

课堂笔记所说的“从表外键为空时才能删除”，本质上是指当从表中不再有非空外键引用这个主表记录时，主表记录才可以删除。

MySQL 还允许在建外键时选择引用动作，例如：

- `ON DELETE RESTRICT`：仍有引用时阻止删除，也是常见的严格做法。
- `ON DELETE CASCADE`：删除主表记录时，同时删除所有从表关联记录。
- `ON DELETE SET NULL`：删除主表记录时，把从表外键改为 `NULL`，要求外键列允许空。

`CASCADE` 会连带删除大量业务数据，生产环境中应明确理解其影响后再使用。

2.3.6 外键约束的价值

- 阻止从表引用不存在的实体，避免形成孤立数据。
- 阻止主表删除仍被使用的实体，保护引用完整性。
- 把关联检查前置到数据库入口，不再完全依赖应用层反复校验。
- 明确表达实体之间的业务关系，使表结构更容易阅读和维护。

外键不是“只要两列名字相似”就会生效。只有真正声明 `FOREIGN KEY ... REFERENCES ...`，数据库才会执行引用检查。

2.4 综合案例：商城表结构

2.4.1 业务需求

课件要求设计一个商店数据库，记录客户、商品和购买行为，包含三张表：

表	主要字段	业务职责
`goods`	商品编号、商品名、单价、分类、供应商	保存商品信息
`customer`	客户编号、姓名、地址、邮箱、性别、身份证	保存客户信息
`purchase`	订单号、客户编号、商品编号、购买数量	记录购买关系

设计要求：

- 每张表都要有主键，关联表要有外键。
- 客户姓名不能为空。
- 邮箱不能重复。
- 客户性别只能从“男、女”中选择。

2.4.2 需求到约束的映射

业务要求	采用的约束
每张表都需要唯一标识	主键
商品、客户、订单需要自动编号	`PRIMARY KEY AUTO_INCREMENT`
客户姓名、商品名称、供应商不能为空	`NOT NULL`
购买数量有合理默认值	`DEFAULT 0`
邮箱、身份证不能重复	`UNIQUE KEY`
性别只能二选一	`ENUM('男', '女') NOT NULL`
购买记录必须引用真实客户和商品	两个外键

这里同时使用了数据类型与表约束：`ENUM` 限制性别可选值，`NOT NULL` 限制缺失，`UNIQUE KEY` 限制重复，外键限制跨表引用。

2.4.3 创建数据库和三张表

```
CREATE DATABASE IF NOT EXISTS bit32mall
DEFAULT CHARACTER SET utf8;

USE bit32mall;

CREATE TABLE IF NOT EXISTS goods (
    goods_id INT PRIMARY KEY AUTO_INCREMENT COMMENT '商品编号',
    goods_name VARCHAR(32) NOT NULL COMMENT '商品名称',
    unitprice INT NOT NULL DEFAULT 0 COMMENT '单价，单位分',
    category VARCHAR(12) COMMENT '商品分类',
    provider VARCHAR(64) NOT NULL COMMENT '供应商名称'
);

CREATE TABLE IF NOT EXISTS customer (
    customer_id INT PRIMARY KEY AUTO_INCREMENT COMMENT '客户编号',
    name VARCHAR(32) NOT NULL COMMENT '客户姓名',
    address VARCHAR(256) COMMENT '客户地址',
    email VARCHAR(64) UNIQUE KEY COMMENT '电子邮箱',
    sex ENUM('男', '女') NOT NULL COMMENT '性别',
    card_id CHAR(18) UNIQUE KEY COMMENT '身份证'
);

CREATE TABLE IF NOT EXISTS purchase (
    order_id INT PRIMARY KEY AUTO_INCREMENT COMMENT '订单号',
    customer_id INT COMMENT '客户编号',
    goods_id INT COMMENT '商品编号',
```

```
    nums INT DEFAULT 0 COMMENT '购买数量',
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id),
    FOREIGN KEY (goods_id) REFERENCES goods(goods_id)
);
```

执行顺序不能随意调换：`purchase` 引用 `customer` 和 `goods`，所以两张主表必须先存在。

2.4.4 表间关系

主表	被引用字段	从表	外键字段	关系含义
`customer`	`customer_id`	`purchase`	`customer_id`	一条购买记录属于某个客户
`goods`	`goods_id`	`purchase`	`goods_id`	一条购买记录对应某个商品

`purchase` 同时引用两张主表，因此它是连接客户与商品的中间表。外键保证每一条购买记录的客户和商品都能在主表中找到，不会出现“不存在的客户购买不存在的商品”这类脏数据。

2.4.5 综合案例的工程改进

示例保留了 `customer_id` 和 `goods_id` 可以为空的写法，用于展示完整约束组合。如果一条购买记录在业务上必须同时拥有客户和商品，可以进一步收紧为：

```
ALTER TABLE purchase
MODIFY customer_id INT NOT NULL,
MODIFY goods_id INT NOT NULL;
```

还可以根据业务增加约束：

- `nums` 至少为 0，或至少为 1。
- `unitprice` 使用整数保存“分”，避免浮点金额误差。
- 邮箱和身份证建立 `UNIQUE NOT NULL`，如果业务要求它们必须填写。
- 新项目通常优先使用 `utf8mb4`，以便完整支持扩展 Unicode 字符。
- 如果需要一次购买多个不同商品，应拆分“订单表”和“订单明细表”，而不是把多个商品塞进一条购买记录。

2.5 第二部分小结

2.5.1 三类约束对照

约束	核心目标	关键特点	常见误用
`AUTO_INCREMENT`	自动生成整数编号	必须为整数且已建索引，每表最多一个	把自增编号当成连续无缺口的业务单据号
`UNIQUE KEY`	保证业务值或组合值不重复	可多个，允许 `NULL`，可复合	以为唯一键自动禁止所有空值
`FOREIGN KEY`	保证跨表引用有效	从表引用主表主键或唯一键，可为 `NULL`	只建立同名字段，没有真正声明外键

2.5.2 表约束的设计顺序

1. 先按业务确定实体和字段。
2. 为每个实体选择稳定、非空的主键。
3. 判断主键是否适合使用 `AUTO_INCREMENT`。
4. 对邮箱、手机号、工号等业务字段建立唯一键。
5. 先创建被引用的主表，再创建带外键的从表。
6. 检查外键的删除策略，避免误删主表数据或留下孤立记录。
7. 用真实业务数据测试边界：空值、重复值、非法引用和删除操作。

2.5.3 核心结论

- `AUTO_INCREMENT` 解决编号生成问题，但不保证编号连续。
- 唯一键解决业务去重问题，允许 `NULL` 并不削弱唯一性本身。
- 主键、唯一键和外键承担不同职责，组合起来才能形成完整的数据约束体系。
- 外键既表达表间关系，也把关系校验交给数据库执行。
- 综合案例把数据类型、非空、默认值、自增、唯一键和外键放进同一套业务模型中。

> 好的表结构不仅记录数据，还让数据库主动拒绝不符合业务关系的写入和删除。